```
  }
}
```

We now need to instruct Catalog Service to fetch its configuration from the Config Service. You can do that via the `spring.config.import` property by passing `configserver:` as the property value. You probably don't want the config server running in your local environment when working with client applications like Catalog Service, and in that case you can make the interaction optional with the `optional:` prefix (`optional:configserver:`). If the config server is not running when you start Catalog Service, the application will log a warning, but it won't stop working. Be careful not to make this optional in production, or you'll risk using the wrong configuration.

Next, Catalog Service needs to know the URL to contact the Config Service. You have two options. Either you can add it to the `spring.config.import` property (`optional:configserver:http://localhost:8888`) or rely on the more specific `spring .cloud.config.uri` property. We'll use the second option so we only need to change the URL value when deploying the application in different environments.

Since the config server uses the application name to return the correct configuration data, you'll also need to set the `spring.application.name` property to `catalog-service`. Remember the `{application}` parameter? That's where the `spring .application.name` value is used.

Open the application.yml file for your Catalog Service project and apply the following configuration.